

Reporte de Práctica - Fase 1: Introducción a React con Vite

Introducción

En esta práctica se desarrolló una aplicación básica utilizando la biblioteca React en conjunto con la herramienta de compilación Vite, con el propósito de comprender de manera práctica la estructura de un proyecto moderno de React, el funcionamiento de los componentes como unidades reutilizables de interfaz, y los dos mecanismos principales para el manejo de datos dentro de un componente: las Props y el State. Vite se utilizó como entorno de desarrollo debido a las ventajas que ofrece frente a herramientas tradicionales como Create React App, entre ellas un arranque casi instantáneo del servidor de desarrollo y un sistema de recarga en caliente (Hot Module Replacement) que permite observar los cambios del código de forma inmediata en el navegador.

Adicionalmente, se analizó el flujo de renderizado de la aplicación, es decir, la secuencia mediante la cual el documento `index.html` carga el punto de entrada en `main.tsx`, el cual monta el componente raíz `App.tsx` dentro del árbol del DOM. Finalmente, se generó la versión de producción del proyecto mediante el comando `'npm run build'`, con el fin de comprender el proceso de compilación y empaquetado que realiza Vite para optimizar la aplicación antes de su despliegue.

Desarrollo

Para alcanzar los objetivos planteados, la práctica se desarrolló siguiendo la secuencia de actividades que se describe a continuación. Cada actividad incluye, al final de su explicación, un espacio destinado a la evidencia fotográfica correspondiente.

1. Creación del proyecto con Vite

Se creó un nuevo proyecto de React ejecutando el comando `'npm create vite@latest'` desde la terminal. Durante el asistente de configuración se seleccionó el framework React junto con la variante TypeScript, lo cual permite aprovechar el tipado estático durante el desarrollo, reduciendo errores en tiempo de compilación y mejorando el autocompletado del editor. Vite generó automáticamente la estructura base del proyecto, incluyendo los archivos de configuración necesarios (`vite.config.ts`, `tsconfig.json`) y el esqueleto inicial de la aplicación.

2. Instalación de dependencias

Una vez generada la estructura del proyecto, se accedió a la carpeta creada y se instalaron las dependencias necesarias mediante el comando `'npm install'`. Este proceso descargó los paquetes especificados en el archivo `package.json` y los almacenó dentro de la carpeta `node_modules`, generando además el archivo `package-lock.json` para fijar las versiones exactas de cada dependencia y garantizar la reproducibilidad del entorno de desarrollo.

3. Ejecución del servidor de desarrollo

Con las dependencias instaladas, se inició el servidor de desarrollo local mediante el comando `'npm run dev'`. Vite levantó el servidor en cuestión de milisegundos y expuso la aplicación en una dirección local, permitiendo visualizar en el navegador la plantilla inicial generada por defecto. Se aprovechó esta etapa para comprobar el funcionamiento del Hot Module Replacement, observando cómo los cambios realizados en el código se reflejaban de inmediato en el navegador sin perder el estado de la aplicación.

4. Análisis de la estructura del proyecto

Se examinó la estructura de carpetas y archivos generada por Vite con el fin de comprender el propósito de cada elemento. El archivo `index.html` funciona como punto de entrada de la aplicación y contiene el elemento contenedor (root) en el que React inyecta el árbol de componentes. El archivo `main.tsx` es el punto de entrada de TypeScript, encargado de montar el componente principal dentro del DOM. El componente `App.tsx` concentra la lógica principal de la interfaz. La carpeta `src` almacena el código fuente de la aplicación, la carpeta `public` contiene los recursos estáticos que no requieren procesamiento, y `node_modules` almacena las dependencias instaladas.

5. Flujo de renderizado de React

Se explicó y documentó el flujo mediante el cual React renderiza la aplicación en el navegador: `index.html` carga el script `main.tsx`, el cual utiliza la API `createRoot` de ReactDOM para tomar el elemento con id `'root'` del documento HTML y montar en él el componente `App.tsx`. A partir de `App.tsx` se renderizan de manera jerárquica el resto de los componentes de la aplicación, formando un árbol de componentes en el que los datos fluyen de manera unidireccional desde los componentes padres hacia los componentes hijos.

6. Creación del componente Saludo.tsx

Se creó un nuevo componente funcional denominado `Saludo.tsx` dentro de la carpeta `src`, con el objetivo de practicar la modularización de la interfaz. Posteriormente, dicho componente fue importado y utilizado dentro de `App.tsx`, evidenciando el principio de reutilización de componentes propio de React, en el cual una misma pieza de interfaz puede ser instanciada múltiples veces con distintos datos de entrada.

7. Implementación de Props

Se implementó el paso de Props hacia el componente `Saludo`, iniciando con el envío del dato `'nombre'` desde `App.tsx`. Posteriormente se amplió la interfaz de Props para incluir también el dato `'tipo'`, reforzando el concepto de que las Props son de solo lectura desde la perspectiva del componente hijo y que su función es transportar información desde un componente padre hacia un componente hijo de forma unidireccional. Este ejercicio permitió comprender cómo React posibilita construir interfaces dinámicas y configurables a partir de datos externos al componente.

8. Implementación del Hook `useState`

Se implementó el Hook `useState` dentro de un componente para crear un contador numérico que se incrementa cada vez que el usuario presiona un botón. Este ejercicio permitió comprender la diferencia fundamental entre Props y State: mientras que las Props son proporcionadas por un componente externo y no pueden modificarse desde dentro del componente que las recibe, el State es un dato interno y mutable, cuyo cambio provoca automáticamente un nuevo renderizado del componente para reflejar el valor actualizado en la interfaz.

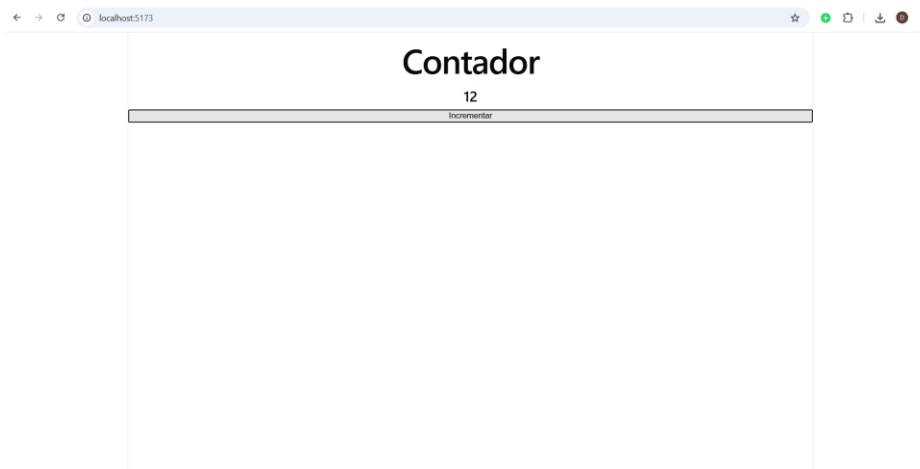
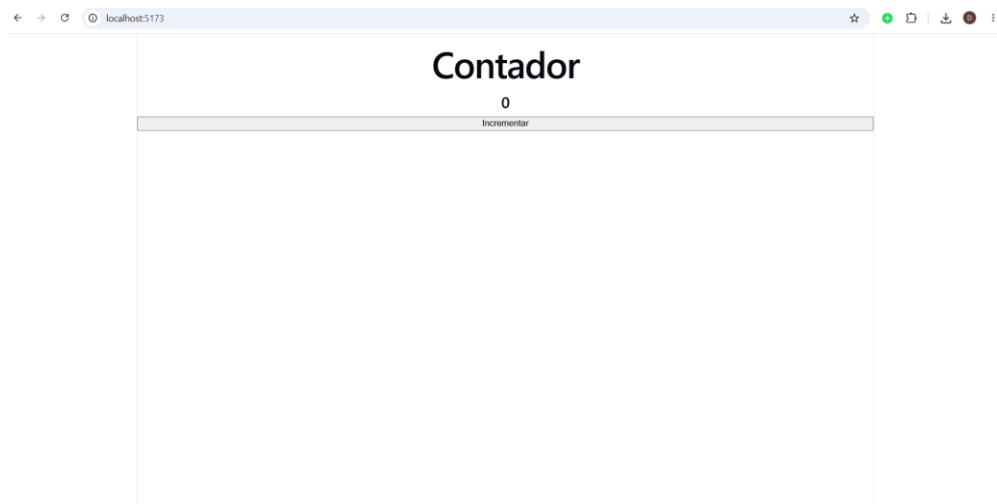
9. Compilación de la aplicación

Finalmente, se generó la versión de producción de la aplicación mediante el comando `'npm run build'`. Durante este proceso, TypeScript verificó el tipado del proyecto y Vite, apoyado internamente en Rollup, empaquetó y minificó los archivos JavaScript, CSS y HTML, generando como resultado la carpeta `dist`.

Esta carpeta contiene la versión optimizada de la aplicación, lista para ser desplegada en un servidor de producción.

Resultados

La aplicación se ejecutó correctamente en el navegador durante todas las etapas de la práctica, sin presentar errores en la consola de desarrollo. El componente Saludo.tsx recibió y desplegó de manera correcta los datos enviados mediante Props, confirmando el funcionamiento del flujo unidireccional de datos entre componentes. El contador implementado con el Hook useState incrementó su valor correctamente cada vez que se presionó el botón correspondiente, demostrando el correcto funcionamiento del manejo de estado interno en React. Por último, la compilación del proyecto mediante 'npm run build' se completó sin errores de tipado ni de empaquetado, generando de forma exitosa la carpeta dist con los archivos optimizados para producción.



Conclusión

Durante esta primera fase de la práctica se comprendieron los fundamentos esenciales del desarrollo de aplicaciones con React utilizando Vite como herramienta de compilación. Se identificó la estructura de un proyecto generado con Vite, así como el papel que desempeña cada uno de sus archivos dentro del flujo de renderizado de la aplicación. Asimismo, se comprendió la diferencia entre Props y State como los dos mecanismos principales para el manejo de datos dentro de un componente, y se practicó la creación de componentes reutilizables como una de las bases de la arquitectura de React.

El uso de Vite permitió apreciar las ventajas de las herramientas modernas de desarrollo frontend frente a configuraciones más tradicionales, particularmente en cuanto a velocidad de arranque y recarga en caliente. Los conceptos abordados en esta fase —estructura del proyecto, componentes, Props y State— constituyen la base necesaria para el desarrollo de aplicaciones web más complejas en fases posteriores de la práctica, donde se espera incorporar funcionalidades adicionales como el manejo de rutas, la comunicación con APIs externas y estrategias más avanzadas de administración del estado.